

Robot Operating System 2

Thejas Padubidri Rao

Driverless Taskphase

(of Affiliation)

Formula Manipal

(of Affiliation)

Manipal, Karnataka

xxxxxxxxx@gmail.com

Abstract— This report provides an overview of the Robot Operating System 2 (ROS 2) framework. It covers the foundational elements of the ROS 2 computation graph, including nodes, topics, services, actions, and parameters. Furthermore, it highlights the differences between topics and services, an appropriate use cases for each. Concepts such as custom interfaces, colcon workspace architecture, and Quality of Service (QoS) settings are also explored. Finally, the report presents the two task solutions accompanied by their respective ROS computation graphs.

Keywords—ROS2, topic, node, colcon, Ubuntu

I. INTRODUCTION

The Robot Operating System 2 (ROS 2) is a framework for writing robot software. It breaks down complex robotic systems into smaller, modular components that communicate with each other. This report outlines the core concepts of the ROS 2 computation graph and workspace architecture, documenting the various core concepts and practical implementations.

II. ROS2 CONCEPTS

A. Computation Graph

The ROS 2 computation graph is a logical representation of all the nodes in a ROS 2 system and how they communicate with each other. The ROS graph is a network of ROS 2 elements processing data together at the same time. It encompasses all executables and the connections between them if you were to map them all out and visualize them. It helps visualize various nodes that are running at any point in time and the publisher or subscriber relationship between them using arrow flows, labelled with the respective topic or service names.

1) Commands

a) To view graphs:

```
ros2 run rqt_graph rqt_graph
```

b) To run rqt to view graphs, logs, etc but without previous configuration:

```
rqt --clear-config
```

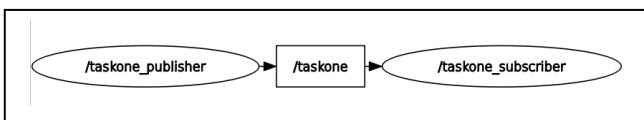


Fig. 1. Sample ROS Graph

B. Nodes

A node in ROS can be considered as a process that is responsible for a single, modular purpose, e.g. controlling the wheel motors or publishing the sensor data from a laser range-

finder. Each node can send and receive data from other nodes via topics, services, actions, or parameters.

1) Commands

a) Launch executable from package

```
ros2 run <package_name> <executable_name>
```

b) List all running nodes

```
ros2 node list
```

c) Remapping/Reassigning default node properties

```
ros2 run <package_name> <executable_name> --ros-args -args
```

d) Get more information about a node

```
ros2 node info <node_name>
```

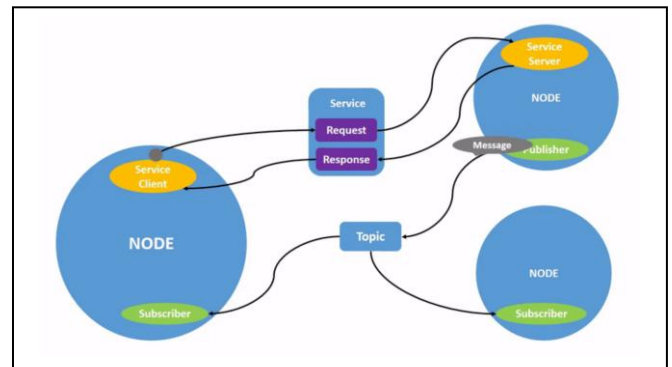


Fig. 2. Representation of Nodes

C. Topics

ROS 2 breaks complex systems down into many modular nodes. Topics are elements of the ROS graph that act as a bus for nodes to exchange messages. A node may publish data to any number of topics and simultaneously have subscriptions to any number of topics. Topics are one of the main ways in which data is moved between nodes and therefore between different parts of the system.

1) Commands

a) See data being published on a topic

```
ros2 topic echo <topic_name>
```

b) Get information on a topic

```
ros2 topic info <topic_name>
```

c) For more detailed information

```
ros2 topic info <topic_name> --verbose
```

d) To know what structure of data the message expects.

```
ros2 interface show <msg_type>
```

e) Publishing data directly using command line

```

ros2 topic pub <topic_name> <msg_type>
'<args>'
f) View rate at which data is being published
ros2 topic hz <topic_name>
g) View bandwidth used by a topic
ros2 topic bw <topic_name>
h) To list a list of available topics
ros2 topic find <topic_type>

```

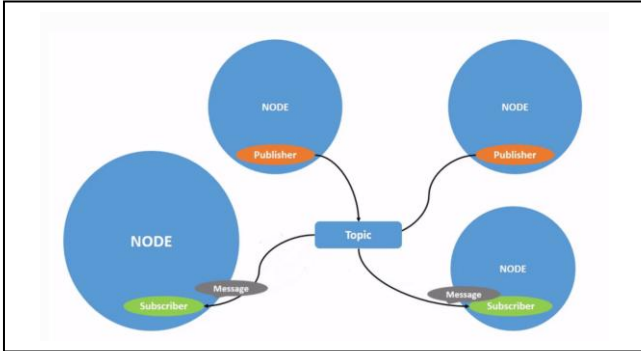


Fig. 3. Representation of Topics

D. Services

Services are another method of communication for nodes in the ROS graph. Services are based on a call-and-response model versus the publisher-subscriber model of topics. While topics allow nodes to subscribe to data streams and get continual updates, services only provide data when they are specifically called by a client.

- 1) *Commands*
 - a) *List services*
`ros2 service list`
 - b) *To know how request and response data of a service is structured*
`ros2 service type <service_name>`
 - c) *See types of all active services*
`ros2 service list -t`
 - d) *Get services of type_name*
`ros2 interface show <type_name>`
 - e) *Call a service*
`ros2 service call <service_name> <service_type> <arguments>`

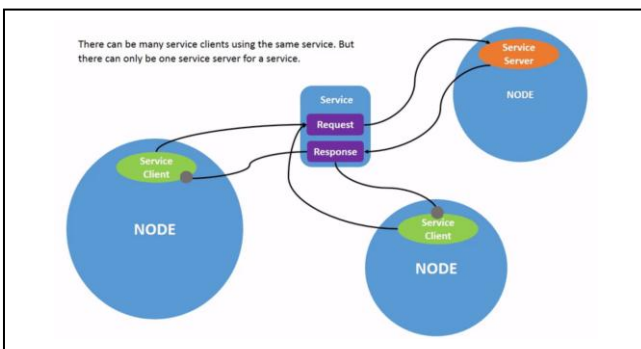


Fig. 4. Representation of Services

E. Actions

Actions are one of the communication types in ROS 2 and are intended for long running tasks. They consist of three parts: a goal, feedback, and a result. Actions are built on topics and services. Their functionality is similar to services, except actions can be canceled. They also provide steady feedback, as opposed to services which return a single response.

Actions use a client-server model, similar to the publisher-subscriber model (described in the topics tutorial). An “action client” node sends a goal to an “action server” node that acknowledges the goal and returns a stream of feedback and a result.

1) *Commands*

- a) *Identify all the actions in the ROS graph*
`ros2 action list`
- b) *Find absolute type*
`ros2 action list -t`
- c) *Introspect action*
`ros2 action info <name>`
- d) *Get action type*
`ros2 interface show <name>`
- e) *Send action goal*
`ros2 action send_goal <action_name> <action_type> <values>`

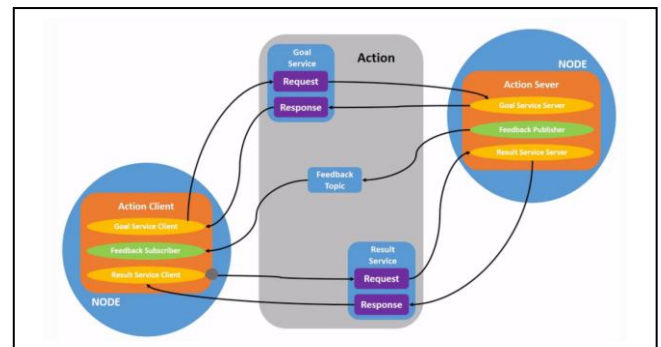


Fig. 5. Representation of Actions

F. Parameters

A parameter is a configuration value of a node. You can think of parameters as node settings. A node can store parameters as integers, floats, booleans, strings, and lists. In ROS 2, each node maintains its own parameters.

1) *Commands*

- a) *See the parameters belonging to your node*
`ros2 param list`
- b) *To display the type and current value of a parameter*
`ros2 param get <node_name> <parameter_name>`
- c) *To change a parameter's value at runtime*
`ros2 param set <node_name> <parameter_name> <value>`
- d) *You can view all of a node's current parameter values*
`ros2 param dump <node_name>`
- e) *Load parameters from a file to a currently running node*

ros2 param load <node_name> <parameter_file>

f) To start the same node using your saved parameter values

ros2 run <package_name> <executable_name> --
ros-args --params-file <file_name>

III. DIFFERENCE BETWEEN TOPICS AND SERVICES

Topics use a publisher-subscriber model and are meant for continuous streams of data. In this setup, a node sends messages out to a topic without needing to know who is listening. Any node that needs the data can subscribe to it. This one-way flow makes topics perfect for handling constant sensor updates, like live LiDAR sensor tracking or motor speeds.

Services, on the other hand, use a request-response model. A client node sends a specific request to a server node and waits for a reply. Unlike topics, which are continuous, services are used for quick, one-time actions. They are best for tasks that have a clear beginning and end, like turning on a light, resetting a sensor, or asking for the car's current battery level.

Feature	Topics	Services
Basic Pattern	Publish / Subscribe	Request / Response
Data Flow	One-way, continuous stream	Two-way, back-and-forth
Connections	Many publishers, many subscribers	One server, many clients
Main Purpose	Sharing constant data	Triggering a specific action

TABLE I. DIFFERENCES BETWEEN TOPICS AND SERVICES

IV. WORKSPACE ARCHITECTURE AND CUSTOMIZATION

A. Colcon and Workspace Structure

ROS 2 packages are organized within a dedicated workspace and built using the colcon build tool. A standard ROS 2 workspace consists of several key directories:

- 1) *src/*: The source space where all ROS 2 packages and their respective codes are placed.
- 2) *build/*: The build space where intermediate files are stored during the colcon build process.
- 3) *install/*: The install space where built executables, libraries, and environment setup scripts are placed. Sourcing the *setup.bash* file from this directory overlays the workspace on top of the base ROS 2 installation.
- 4) *log/*: Contains the build logs for debugging compilation errors.

B. Custom Interfaces

While ROS 2 provides standard message types (e.g., *std_msgs*), complex robotic systems often require Custom Interfaces. These are defined in dedicated files:

- 1) *.msg files*: Define custom topic data structures.
- 2) *.srv files*: Define custom service requests and responses separated by a --- line.
- 3) *.action files*: Define custom action goals, results, and feedback separated by --- lines.

Custom interfaces must be declared in the package's *CMakeLists.txt* using the *rosidl_default_generators* function and added as dependencies in the *package.xml* to ensure they

compile correctly into language-specific libraries (C++ and Python).

C. Quality of Service (QoS) Settings

Unlike ROS 1, ROS 2 is built on the Data Distribution Service (DDS), which introduces Quality of Service (QoS) policies to manage how data is communicated under varying network conditions. Key QoS settings include:

- 1) *Reliability*: Defines delivery guarantees. Reliable ensures the message is delivered (retransmitting if necessary), whereas Best Effort sends data once without delivery confirmation (ideal for high-frequency sensor data).
- 2) *History*: Determines how messages are queued. Keep Last only stores up to a specified queue depth (N), while Keep All stores all messages until resource limits are hit.
- 3) *Durability*: Manages late-joining subscribers. Volatile means messages are only available to existing subscribers at the time of publishing. Transient Local saves the most recent messages so that new subscribers can receive them immediately upon connecting.

V. TASK SOLUTIONS AND COMPUTATION GRAPHS

This section demonstrates the practical implementation of ROS 2 concepts across two assigned tasks. The computation graphs generated using *rqt_graph* illustrate how data moves between the nodes in each solution.

A. Task 1 Solution: Longitudinal Speed Calculator

A custom message named *TaskOne* was created to hold two numerical values: angular velocity (*angVel*) and wheel radius. The system consists of two nodes:

- 1) *Publisher Node*: This node prompts the user to enter the angular velocity and radius into the terminal. It then bundles this data into the custom message and publishes it to a topic named */taskone*.
- 2) *Subscriber Node*: This node continuously listens to the */taskone* topic. Upon receiving the data, it calculates the longitudinal speed of the wheel ($\text{longitudinal speed} = \text{radius} \times \text{angular velocity}$) and prints the result to the terminal.

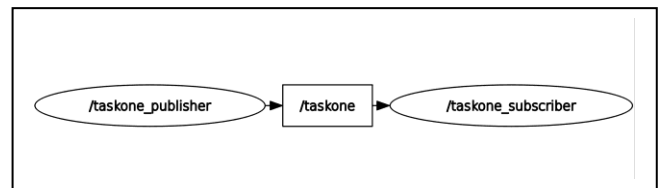


Fig. 6. Graph for Task 1: Longitudinal Speed Calculator

B. Task 2 Solution: Multi-Node Palindrome Checker

The second task demonstrates a more complex data flow involving three interconnected nodes and a custom string message. A custom message named *RT2String* was created to hold string data. The system is split across three distinct nodes:

- 1) *Node 1*: Takes a string input from the user (e.g., "AABAA" or "BACAB") and continuously publishes it to a designated topic.
- 2) *Node 2*: Subscribes to the topic from Node 1. Once it receives the string, it evaluates whether the word is a palindrome (reads the same forwards and backwards). It then

sends a service request to Node 3: a "1" if it is a palindrome, or a "0" if it is not.

3) *Node 3*: Receives the numerical input (1 or 0) from Node 2. It processes this flag and sends a response back to Node 2 "Yes" for 1, and "No" for 0. The final result is then printed on the terminal.

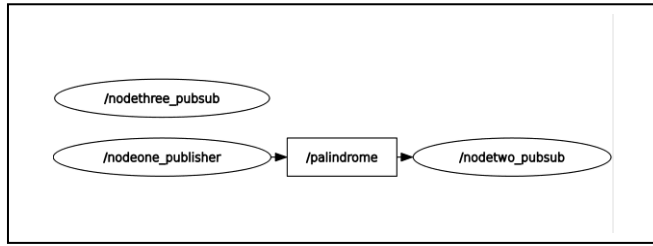


Fig. 7. Graph for Task 2: Multi-Node Palindrome Checker

VI. WORKING WITH ROS2 ON UBUNTU

A. Commands for Installing ROS2

```

locale # check for UTF-8

sudo apt update && sudo apt install locales

sudo locale-gen en_US en_US.UTF-8

sudo update-locale LC_ALL=en_US.UTF-8
LANG=en_US.UTF-8

export LANG=en_US.UTF-8

locale # verify settings

sudo apt install software-properties-common

sudo add-apt-repository universe

sudo apt update && sudo apt install curl -y

export ROS_APT_SOURCE_VERSION=$(curl -s
https://api.github.com/repos/ros-infrastructure/ros-apt-
source/releases/latest | grep -F "tag_name" | awk -F'"' '{print
$4}')

curl -L -o /tmp/ros2-apt-source.deb
"https://github.com/ros-infrastructure/ros-apt-
source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-
source_${ROS_APT_SOURCE_VERSION}.${UBUNTU_CODENAME:-
$(cat /etc/os-release | grep -F "VERSION_CODENAME" | awk -F'"' '{print
$4}')}_all.deb"

sudo dpkg -i /tmp/ros2-apt-source.deb

sudo apt update

sudo apt upgrade

sudo apt install ros-humble-desktop

sudo apt install ros-dev-tools

```

B. Commands to set up ROS2

```

echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc

echo "export ROS_DOMAIN_ID=<your_domain_id>"
>> ~/.bashrc

echo "export ROS_LOCALHOST_ONLY=1" >>
~/.bashrc

```

C. Command to create a workspace

```
mkdir -p ~/<workspace_name>/src
```

D. Command to create a package in C++ in src

```
ros2 pkg create --build-type ament_cmake --license
Apache-2.0 <package_name>
```

E. Command to create a package in Python in src

```
ros2 pkg create --build-type ament_python --license
Apache-2.0 <package_name>
```

F. Command to build all packages

```
colcon build
```

G. Command to build selective packages

```
colcon build --packages-select <package_name>
```

H. Lines to include in package.xml file

```

<build_depend>roscpp</build_depend>
>
<exec_depend>roscpp</exec_depend>
<member_of_group>roscpp</member_of_group>

```

I. Add entry point in setup.py in case of python packages

For example in task one, under entry_points include

```

'console_scripts': [
    "pub = taskone.taskone_publisher:main",
    "sub = taskone.taskone_subscriber:main"
],

```

Format:

```
terminal_command_name=package_name.python_file_n
ame_without_py:main_function
```

VII. CONCLUSION

This report presented the essential concepts of ROS 2, including workspace structure, node initialization, and different communication types. Through the implementation of both tasks, the practical application of custom messages and multi-node architectures was successfully demonstrated. Task 1 verified the use of publisher-subscriber systems for continuous data transmission and calculation, while Task 2 showcased conditional logic and sequential data hand-offs across three separate nodes. The successful execution of these nodes and the generation of their respective ROS 2 package graphs verify a fundamental understanding of ROS 2 package creation and inter-node communication.

ACKNOWLEDGMENTS

I would like to express my gratitude to FORMULA MANIPAL, for the assignment that prompted this deep dive into ROS2. I also acknowledge the creators of the *docs.ros.org* documentation website, which significantly aided in understanding concepts related to ROS2.

REFERENCES

- [1] Open Robotics, "Tutorials – ROS 2 Documentation".